

Individual Assessment Coversheet

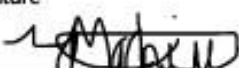
To be attached to the front of the assessment.

Campus:	Bedfordview Campus
Faculty:	Technology
Module Code:	ITJVA 2-12
Group:	1
Lecturer's Name:	Mrs Stacey Byrne
Student Full Name:	Gefte Machivene
Student Number:	CJ8DRDW47

Indicate	Yes	No
Plagiarism report attached	☐	☐

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature 	Date 01/08/2025
---	---------------------------

Lecturer's Comments:

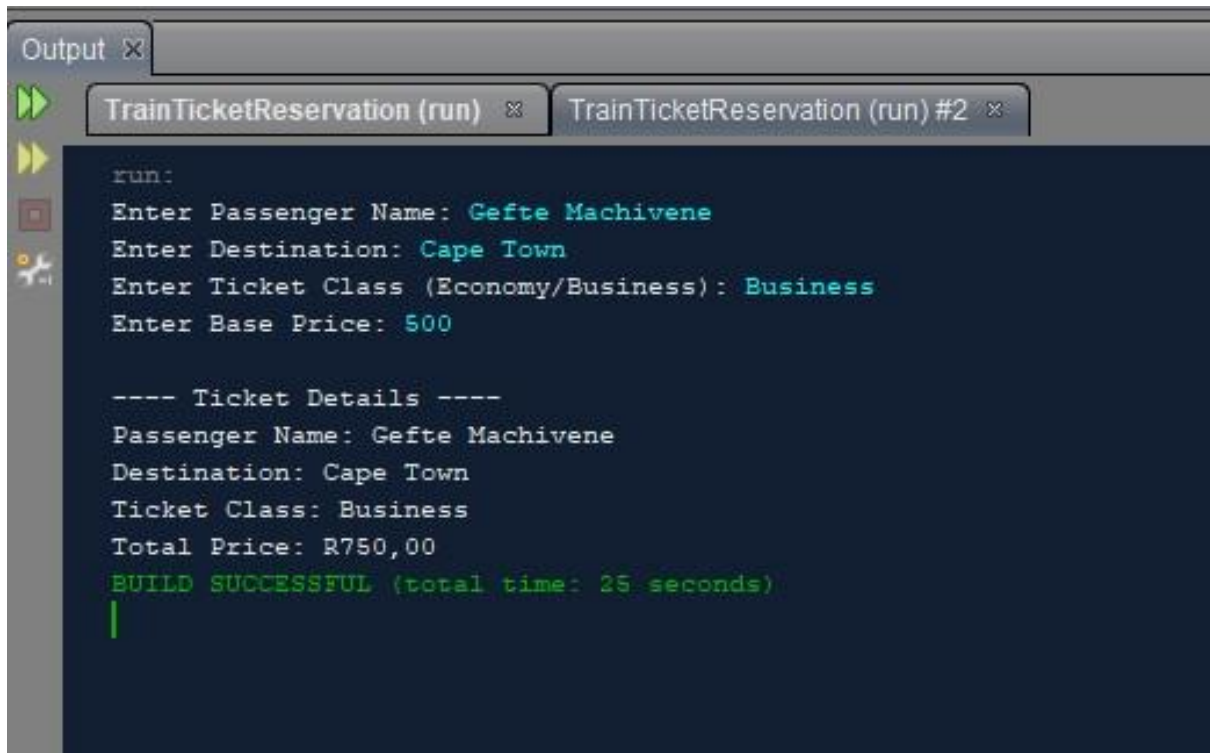
Marks Awarded:	%
-----------------------	---

Signature	Date
------------------	-------------

Table of Contents

QUESTION 1	3
Question 2.....	4
Question 3.....	6
References	8

QUESTION 1



The screenshot shows a Java IDE with two tabs: "TrainTicketReservation (run)" and "TrainTicketReservation (run) #2". The output window displays the following text:

```
run:
Enter Passenger Name: Gefte Machivene
Enter Destination: Cape Town
Enter Ticket Class (Economy/Business): Business
Enter Base Price: 500

---- Ticket Details ----
Passenger Name: Gefte Machivene
Destination: Cape Town
Ticket Class: Business
Total Price: R750,00
BUILD SUCCESSFUL (total time: 25 seconds)
```

A program in the main() method that allows the user to:

- Enter passenger name, destination, price and ticket class
- Use if statements or switch statements to validate the ticket class input
- Apply price multipliers based on ticket class:
 - **Economy** = base price
 - **Business** = base price × 1.5

The program then runs to print out the ticket information.

Question 2

The screenshot shows a window titled "Train Ticket System". It contains four input fields with labels above them: "PASSENGER NAME" (containing "Luffy"), "DESTINATION" (containing "Cape Town"), "TICKET CLASS (Economy/Business)" (containing "Economy"), and "PRICE (R)" (containing "500"). Below these fields are two buttons: "ADD TICKET" and "SEARCH DESTINATION". At the bottom, there is a section titled "TICKET DETAILS" which is currently empty.

Designed a GUI that is based question 1 logic. The GUI gets the user details and once they enter the details. It prints it out to the Ticket Details.

This screenshot shows the same "Train Ticket System" window. The input fields and buttons are identical to the previous image. However, the "TICKET DETAILS" section is now populated with the following text: "Passenger: Luffy", "Destination: Cape Town", "Class: Economy", and "Price: R500,00". A dashed line is visible below the price.

Train Ticket System

PASSENGER NAME

DESTINATION

TICKET CLASS (Economy/Business)

PRICE (R)

ADD TICKET

SEARCH DESTINATION

TICKET DETAILS

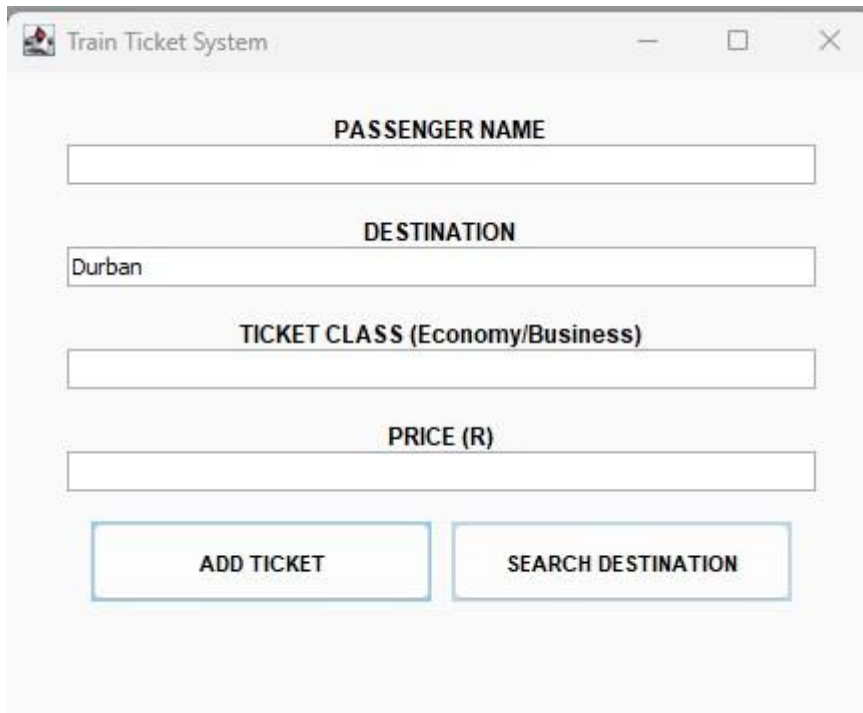
Passenger: Luffy
Destination: Cape Town
Class: Economy
Price: R500,00

Passenger: Zoro
Destination: Durban
Class: Business
Price: R1125,00

Passenger: Nami
Destination: Durban
Class: Economy
Price: R300,00

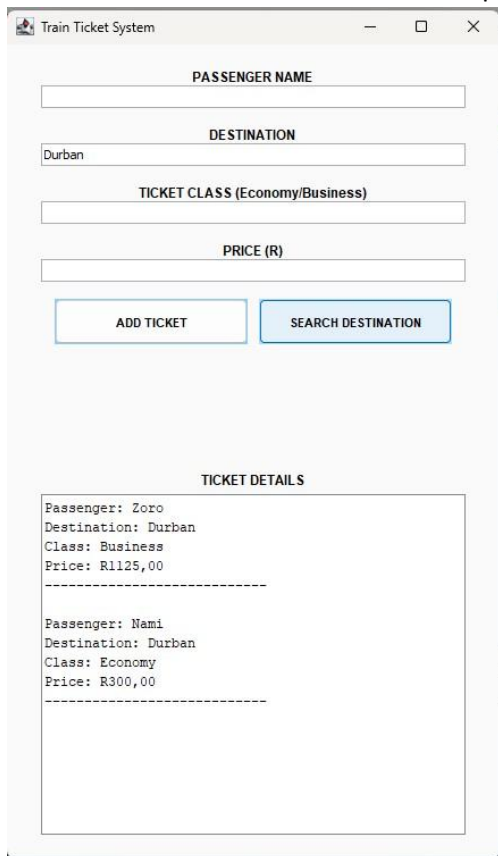
Added more details to show how the details would look when more data is loaded

Question 3



The screenshot shows a window titled "Train Ticket System". It contains four input fields with labels above them: "PASSENGER NAME", "DESTINATION", "TICKET CLASS (Economy/Business)", and "PRICE (R)". The "DESTINATION" field contains the text "Durban". Below the input fields are two buttons: "ADD TICKET" and "SEARCH DESTINATION".

To search for a ticket, a user needs to input the destination they want to find tickets of. After clicking the Search Destination Button. The system will search for all the tickets in the ArrayList that have the same destination. The results will be displayed in the Ticket Display area.



The screenshot shows the same "Train Ticket System" window, but now it displays search results in a "TICKET DETAILS" section. The input fields and buttons remain the same. The "TICKET DETAILS" section contains two entries, each separated by a dashed line:

```
Passenger: Zoro
Destination: Durban
Class: Business
Price: R1125,00
-----
Passenger: Nami
Destination: Durban
Class: Economy
Price: R300,00
-----
```

The SQL Database that stores ticket details that is entered from the GUI. The tickets_ID auto increments with each new entry

The username = root

The password = #Gmachivene18

The screenshot displays a SQL query editor window titled 'Query 1'. The query contains the following SQL statements:

```
1 • CREATE DATABASE trainDB;
2 • USE trainDB;
3 • CREATE TABLE Tickets (
4     ticket_id INT AUTO_INCREMENT PRIMARY KEY,
5     passenger_name VARCHAR(100),
6     destination VARCHAR(100),
7     ticket_class VARCHAR(50),
8     price DOUBLE
9 )
10 SELECT * FROM trainDB.Tickets;
```

Below the query editor is the 'Result Grid' showing the output of the query. It contains 6 rows of data:

	ticket_id	passenger_name	destination	ticket_class	price
▶	1	Gefte Machivene	Cape Town	Business	750
	2	Stacy Java	Durban	Economy	200
	3	Khumo	East London	Economy	50
	4	Luffy	Cape Town	Economy	500
	5	Zoro	Durban	Business	1125
	6	Nami	Durban	Economy	300
*	NULL	NULL	NULL	NULL	NULL

At the bottom of the screenshot is the 'Output' section, which shows the 'Action Output' for the query. It indicates that the query 'SELECT * FROM trainDB.Tickets LIMIT 0, 1000' was executed at 14:22:54 and returned 3 row(s).

References

- **Bro Code.** 2023. *Java Full Course for Beginners*. [Video]. Available at: https://www.youtube.com/watch?v=xk4_1vDrzzo [Accessed 21 February 2026].
- **W3Schools.** 2024. *Java User Input (Scanner)*. [Online]. Available at: https://www.w3schools.com/java/java_user_input.asp [Accessed 09 March 2026].
- **Oracle Docs.** 2025. *The List Interface (ArrayLists explained)*. [Online]. Available at: <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html> [Accessed 09 March 2026].
- **GeeksforGeeks.** 2025. *Formatted output in Java*. [Online]. Available at: <https://www.geeksforgeeks.org/formatted-output-in-java/> [Accessed 09 March 2026].
- **Programiz.** 2024. *Java Try-Catch Block*. [Online]. Available at: <https://www.programiz.com/java-programming/try-catch> [Accessed 09 March 2026].
- **Coding with John.** 2024. *Java Exception Handling (Try-Catch)*. [Video]. Available at: <https://www.youtube.com/watch?v=1XAfapkBQjk> [Accessed 12 March 2026].
- **Coders Campus.** 2024. *How to use ArrayLists in Java*. [Video]. Available at: <https://www.youtube.com/watch?v=5I3pYpX-u60> [Accessed 12 March 2026].
- **SQLBolt.** 2023. *Introduction to SQL: Creating Tables*. [Online]. Available at: https://sqlbolt.com/lesson/creating_tables [Accessed 15 March 2026].
- **Keep On Coding.** 2023. *Java JDBC Tutorial - Connecting to a Database*. [Video]. Available at: <https://www.youtube.com/watch?v=2i4t-Chvm2A> [Accessed 15 March 2026].
- **TutorialsPoint.** 2024. *JDBC - Insert Records Example*. [Online]. Available at: <https://www.tutorialspoint.com/jdbc/jdbc-insert-records.htm> [Accessed 15 March 2026].
- **ProgrammingKnowledge.** 2023. *Java Swing GUI Tutorial with NetBeans*. [Video]. Available at: <https://www.youtube.com/watch?v=Km8vOnLOV6A> [Accessed 13 March 2026].
- **MySQL Tutorial.** 2025. *MySQL CREATE TABLE Statement*. [Online]. Available at: <https://www.mysqltutorial.org/mysql-create-table/> [Accessed 18 March 2026].